

SplitInfer: Moving Memory to Compute, or Compute to Memory?

A Measured Crossover for Edge Inference Beyond SoC Memory

Hemraj Lamkuche

Abstract—When a model’s weights exceed an edge SoC’s unified memory, the system has two choices: move the memory to the compute, or move the compute to the memory. Which is cheaper is not a property of the accelerator; it is a property of the interconnect. This paper builds *both* paths over one substrate and measures where they cross. The substrate is a Jetson Orin Nano attached to a commodity FPGA memory node (Nexys 4 DDR, 128 MB DDR2) by a deliberately slow UART link. CXLWIN maps the FPGA’s DDR2 into the host address space so that ordinary loads and stores reach device memory through page faults, with an explicit host-bias/device-bias ownership model in the manner of CXL.mem Type 2 devices—stated as a page-granular software analogy, not as equivalence. Near-memory engines on the FPGA execute INT8 embedding gathers, an 8×8 MAC array, and a fused requantisation epilogue. An INT8 lowering produces a single program that executes in three modes—host-resident, pool (memory→compute), and NMC (compute→memory)—with bit-exact integer agreement across all three, and an exact dynamic-programming placer prices all three per layer. On a 36-node DLRM the emulated-link sweep shows NMC winning below $\sim 10^8$ bytes/s and pool winning above; measured on the board at the UART point, NMC costs 546 ms per inference against 1.64 ms host-resident and a cold pool costs 309,573 ms, both bit-exact against the host reference and within 1% of the emulator’s prediction. A 167.6 MiB INT8 model larger than the FPGA’s memory executes end to end through two rotating 23.9 MiB slots with a 48.0 MiB peak footprint—and a stated price of 32,631 ms per inference of weight streaming at USB 2.0 rates. Every claim is labelled measured, emulated, or modelled.

Index Terms—edge inference, memory disaggregation, near-memory computing, software distributed shared memory, FPGA, CXL, model partitioning, INT8 quantisation.

I. INTRODUCTION

AN edge SoC such as the NVIDIA Jetson Orin Nano offers 8 GB of LPDDR5 shared by CPU, GPU and operating system. Models that do not fit are not slow; they thrash, then die. On this platform a dense 6.0 GB FP32 MLP runs 1,982 ms per inference against 273 ms for a 4.58 GB one, and a 9.39 GB model is OOM-killed during load (Fig. 1). Once capacity is the constraint, a designer has exactly two levers: attach more memory and bring the weights to the SoC’s compute each inference, or put compute next to the extra memory and bring only activations across.

The datacenter answer is CXL: coherent, load/store access to device-attached memory, with the CXL.mem bias model letting a Type 2 device own its memory while it computes on it. No CXL controller exists for edge SoCs. But the two ideas underneath CXL—*host load/store access to remote memory* and *explicit ownership transfer between host and device compute*—do not require CXL silicon. They can be built in software at page granularity, as the distributed-shared-memory systems of the 1990s did over Ethernet [1]–[3]. That is what this paper does, and then uses the result to answer a question the CXL literature rarely asks because its links are fast: *at what link bandwidth does moving compute to memory stop paying?*

Contributions.

- 1) **CXLWIN**, a host load/store window onto FPGA-attached memory (§III-A). Page faults are serviced by a small message protocol (EdgeCoh) over the link; six page states, including two transient states that track outstanding transfers, implement host/device bias ownership. It needs no kernel feature and no special hardware on the host side.
- 2) **A three-placement cost model and an exact dynamic-programming placer** (§III-E) that prices, per layer, host-resident execution, pool execution (fault weights in, compute locally), and near-memory execution, including the weight-streaming traffic that a capacity-unlocking design necessarily incurs, and that yields a closed-form per-layer crossover bandwidth.
- 3) **A verified data path** (§IV): INT8 lowering to a DDR2 image and an executable program; FPGA engines (gather, 8×8 INT8 MAC, fused requantisation) and host kernels with identical integer semantics; a virtual FPGA that consumes the same protocol bytes as the board; and bit-exact agreement across host, pool and NMC modes on every test vector.
- 4) **The crossover, measured by execution** (§VI): the same program run in every mode across six link classes, the capacity mechanism run end to end on a model larger than device memory with its streaming cost stated, and a claims discipline that labels every number measured (^M), emulated (^E) or modelled (^C).

Relation to a prior version. An earlier manuscript on this system reported a 595 ms latency for the FPGA path. Post-

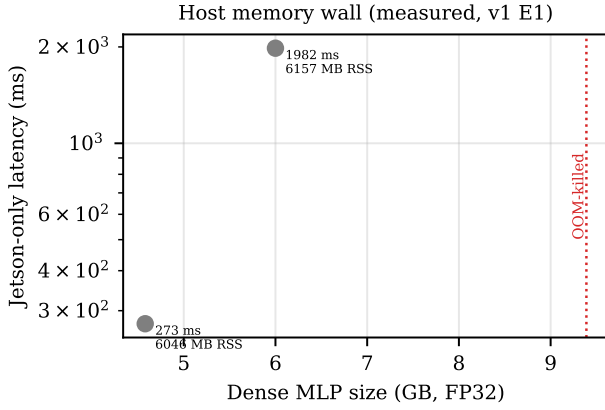


Fig. 1. Jetson-only latency versus dense-model size (measured^M). Latency rises non-linearly at 6 GB and the process is killed at 9.39 GB.

mortem analysis found that its runtime moved no operands (every activation size had been recorded as zero), so that figure timed empty protocol round trips; its partitioner was a greedy pass; its MAC array broadcast one dot product to eight accumulators; and its synthetic models had all-zero weights. None of those numbers appear here. Everything in this paper is produced by scripts in the artifact from the data files they reference.

II. BACKGROUND

A. The memory wall, measured

Fig. 1 shows Jetson-only inference on dense MLPs of increasing size (onnxruntime, FP32). The transition from 273 ms at 4.58 GB to 1,982 ms at 6.0 GB is a $7\times$ latency increase for $1.3\times$ more parameters—page thrashing—and at 9.39 GB the kernel OOM-killer terminates the process with 7,373 MB resident. These are dense models, in which every weight participates in every inference; sparse embedding tables memory-map lazily and do not exhibit the wall. The wall is the regime this paper addresses.

B. Two ways to use attached memory

Given a memory node on a link of bandwidth B and per-message latency r , a layer with W weight bytes and F floating-point operations can execute in one of three placements:

- **host**: weights resident in SoC memory, compute on the GPU; cost F/G_{gpu} ; consumes W of host memory.
- **pool**: weights resident on the node; the host faults them in through CXLWIN and computes locally; cost $W/B + n_{\text{msg}}r + F/G_{\text{gpu}}$ with n_{msg} the number of fault batches; consumes no persistent host memory.
- **NMC**: weights resident on (or streamed to) the node; the node's engine computes; cost $S(W) + r + F/G_{\text{nmc}}$ where S is the streaming cost, zero if the weights are already resident.

Pool moves memory to compute; NMC moves compute to memory. For resident weights the two are equal when

$$B^* = \frac{W}{F/G_{\text{nmc}} + r - F/G_{\text{gpu}} - n_{\text{msg}}r}, \quad (1)$$

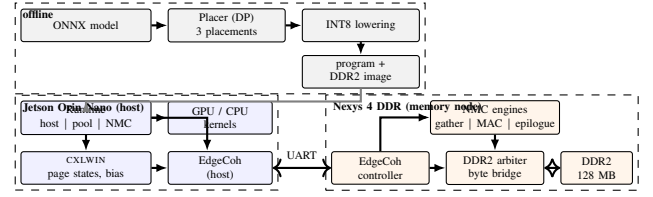


Fig. 2. SplitInfer. Offline: the placer chooses host/pool/NMC per layer and the lowering emits one INT8 program and DDR2 image. Online: the runtime executes the program in a mode; pool mode dereferences pointers into the CXLWIN window, whose page faults become EdgeCoh reads; NMC mode dispatches to the engines and reads back results.

TABLE I
CXLWIN PAGE STATES

State	Protection	Meaning
INVALID	none	no valid host copy; device holds the data
FETCHING	none	<i>transient</i> : a read is outstanding
SHARED	read	host holds a clean copy; device copy valid
MODIFIED	read/write	host copy dirty; device copy stale
FLUSHING	read	<i>transient</i> : a write-back is outstanding
DEVICE	none	device bias: an engine owns the page

a per-layer crossover bandwidth. Above B^* the link is fast enough that hauling weights to the faster GPU wins; below it, the slow engine next to the memory wins because it avoids the haul. §III-E uses (1) inside an exact placer; §VI checks it against executed latencies.

C. CXL.mem as an analogy, software DSM as an ancestor

CXL.mem gives a host cache-line-granular, hardware-coherent loads and stores to device memory; a Type 2 device additionally holds a *bias* per page—host bias, where the host's caches are authoritative, or device bias, where the device may operate on the memory without snooping the host. CXLWIN reproduces the second idea faithfully and the first only at page granularity, in software, without hardware coherence. The mechanism—protect the pages, service faults by fetching over the network, track ownership in a per-page state machine—is that of Ivy [1], Munin [2] and TreadMarks [3]. We claim no novelty in the mechanism. What is new is using it as the *memory-to-compute* arm of a controlled comparison against near-memory compute on the same device, over the same link, with the same program.

III. DESIGN

A. CXLWIN: a load/store window with ownership

CXLWIN reserves a host virtual range the size of the device region with no access permission. A dereference faults; a handler that uses only async-signal-safe calls hands the address to a service thread, which performs the transfer, changes the page protection, and releases the faulting thread to re-execute the instruction. A store to a readable page faults a second time and upgrades the page to writable and dirty. Table I lists the six page states and Fig. 3 the transitions.

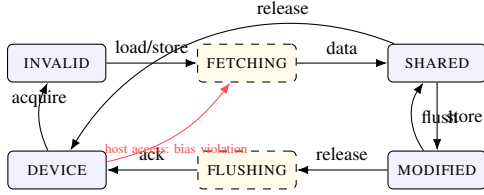


Fig. 3. Page-state transitions. Dashed states are transient and exist only while a transfer is outstanding. A host access to a DEVICE-bias page is a coherence violation; by default it is counted and serviced, in strict mode it aborts.

Ownership. `release(range)` writes back MODIFIED pages and moves the range to DEVICE; the runtime calls it before dispatching an engine on that memory. `acquire(range)` moves DEVICE pages to INVALID; the next host access re-fetches whatever the engine wrote. This is the CXL Type 2 bias flip, at page granularity and directed by software. The transient states matter for correctness: a second fault on a page whose read is outstanding must wait for that read, not issue another, and a `release` racing a fault must not write back a half-filled page. Both are guaranteed by holding the window’s lock across the transient state.

Prefetch. A miss fetches P contiguous pages in one message. $P=1$ is demand paging; larger P amortises the per-message latency r but fetches pages that may never be touched. §VI shows that this trade-off has a sign that depends on the layer.

What CXLWIN is not. It is not cache-line coherent, it does not snoop, and it does not make the device’s memory appear in the host’s physical address map. Its guarantee is weaker and explicit: between `release` and `acquire` the device’s view is authoritative; outside that interval the host’s view is. That is enough for layer-sequential inference and is stated as such.

B. EdgeCoh: the message protocol

EdgeCoh carries CXLWIN’s faults and the runtime’s dispatches. Every message has an 8-byte header (type, flags, tensor id, payload length). `DATA_WRITE` and `DATA_READ` move bytes to and from DDR2 byte addresses; `NMC_EXEC` carries an opcode and six 32-bit parameters; `SYNC_BARRIER`, `TRANSFER_OWNERSHIP` and `PREFETCH` are one-round-trip control messages; the device answers everything with `ACK` or `DATA_RESPONSE`. The protocol is deliberately dumb: the intelligence is in CXLWIN’s state machine and the placer, not in the wire format.

C. Near-memory engines

Three engines operate on DDR2-resident INT8 tensors in the memory controller’s clock domain (81.25 MHz): *gather* copies rows of an embedding table selected by INT32 indices; *INT8 FC* computes $y = Wx$ for a row-major W of $M \times K$ INT8 weights into INT32 accumulators with an 8×8 array—eight independent lanes, each an 8-wide dot product, so that one 16-byte activation read is reused across eight weight rows; *epilogue* computes $\text{sat}_8(\text{relu}((y+b) \cdot m \gg s))$ with an INT32 bias b , a 16-bit multiplier m and a shift s . The epilogue is the standard fixed-point requantisation, chosen precisely because

a host can reproduce it bit for bit. Alignment rules (16-byte addresses, K padded to 16, M to 16) are imposed by the lowering, not the user.

D. INT8 lowering and the program

The lowering takes an ONNX graph and a placement and produces a *program*—an ordered list of layer records with DDR2 addresses, dimensions and requantisation parameters—a DDR2 *image* of INT8 tables and weights and INT32 biases, and a set of *test vectors* with expected integer outputs. Activation scales are calibrated per tensor on random inputs through `onnxruntime`; `MATMUL`→`ADD`→`RELU` chains fuse into one FC record; `CONCAT` is made free by pinning each producer’s output inside the concatenation buffer. The same program executes in every mode, and every mode is checked against the same expected integers.

E. Placement

The placer is an exact dynamic program over (layer, placement, host-memory-used, device-memory-used), with memory tracked in adaptive buckets and an argmin table for backtracking. The cost of a layer in a placement is the sum in §II; a change of compute location between consecutive layers adds an activation crossing. Two constraints bound it: host-resident weights must fit a host budget, and device-resident weights must fit device memory—either individually (streamed) or cumulatively (resident). The DP is verified against exhaustive enumeration on small instances. Its parameters are meant to be measured: the artifact ships a calibration script that fits B and r from the link, and the engine rate comes from cycle-accurate simulation of the MAC data path (0.684 MAC/cycle, 0.111 GOPS at 81.25 MHz).

The placer operates on the ONNX graph, before fusion. Its per-layer decisions and crossover bandwidths are reported (§VI); the runtime in this paper executes whole programs in one mode at a time, and executing a mixed placement layer by layer is future work.

IV. IMPLEMENTATION AND VERIFICATION

FPGA. The design runs in two clock domains bridged by handshake CDCs and gray-code FIFOs: a 100 MHz domain for the UART and the EdgeCoh controller, and the MIG’s 81.25 MHz domain for a DDR2 arbiter and the engines. The arbiter owns the MIG user interface, converts byte addresses to the controller’s 16-bit-word address space, holds commands until the controller accepts them, tags the single outstanding read with its owner, queues two writes per engine port, and bridges the controller’s byte stream to 128-bit bursts with a byte-masked write assembler and a one-word read cache that preserves read-after-write ordering.

Host. CXLWIN and EdgeCoh are C; the lowering and placer are Python; the runtime and kernels are C++. The runtime’s three executors share one set of integer kernels: the host executor runs them on host-resident weights, the pool executor runs the same kernels on pointers into the CXLWIN window, and the NMC executor sends dispatches.

Virtual FPGA. The EdgeCoh transport is a small vtable, and a second implementation of it is an in-process device model: 128 MB of DDR2 as a byte array and the three engines with the RTL’s integer semantics, plus an emulated link (bandwidth, per-message latency) and emulated engine time (cycle counts from simulation). Everything above the transport—runtime, CXLWIN, lowering—is byte-for-byte identical between the emulator and the board. Numbers from the emulator are labelled ^E and are never presented as measurements.

Verification. Four testbenches under Icarus Verilog drive the RTL with random operands against reference computations—a DMA byte round trip through unaligned addresses with guard bytes, a 24×48 INT8 matrix-vector product, a 32-row gather, and a 44-element epilogue with a partial tail—and the virtual FPGA passes the same vectors through the protocol API. The lowering’s reference executor, the host executor, the pool executor and the NMC executor are then required to produce identical INT32 outputs on every test vector; a mismatch fails the run. Table II reports that check for every cell.

What golden vectors found. The v1 design had passed functional simulation and a hardware loopback test that compared only message headers. Golden vectors found that its engines passed byte addresses into the memory controller’s word-unit address space, that its byte-stream writes zeroed fifteen neighbouring bytes and its byte reads always returned lane 0 of a burst, that its gather always selected the first index of each 16-byte word, and that its MAC array broadcast one dot product to all eight accumulators. None of these could have been caught by a test that checks acknowledgements. The board then found two more that simulation had passed: the UART transmitter’s ready flag is registered and stays high for one cycle after accepting a byte, so the response-header state emitted every second byte; and the memory controller’s read-data bus is valid only in the cycle its valid flag is high, whereas the behavioural model held the last value—the gather engine copied a cycle late and every embedding row came back wrong while the pool path, which reads through the DMA bridge, was bit-exact. Both were reproduced in simulation only after the model was made as strict as the silicon. We record this as the methodological lesson of the project: for a memory node, the test that matters is the one that reads back what it wrote, and the model must be at least as unforgiving as the hardware about *when* data is valid.

V. METHODOLOGY

Platform. Jetson Orin Nano Super (JetPack 6, R36.5, TensorRT 10.7, CUDA 12.6); Digilent Nexys 4 DDR (XC7A100T, 128 MB DDR2) over an FT2232HQ UART at 115,200 baud, 8N1 (11,520 B/s ceiling). All host-side numbers are taken on the Jetson. The link was calibrated on the board (^M): 11,464 B/s sustained and a 8.0 ms message round trip. The round trip was 16.0 ms until the FTDI bridge’s latency timer—16 ms by default, the interval for which it holds received bytes before forwarding them—was set to 1 ms; the transport now does this on open. Half of every dispatch and every page fault on this substrate was the USB bridge, not the wire.

Workloads. A synthetic DLRM [19]: 26 embedding tables of 1000×64, a 128-feature dense input, and a 1792→1024→256→1 top MLP (36 ONNX nodes, 14.4 MiB FP32, 3.6 MiB INT8). Dense MLPs of 5000×5000 layers for the capacity experiments. Weights are seeded random ($\mathcal{N}(0, 1/\sqrt{\text{fan-in}})$); an earlier version of this work used all-zero weights, which invalidates any correctness check and makes INT8 quantisation degenerate.

Baselines. B1: onnxruntime FP32 on the CPU. B2: onnxruntime dynamic INT8 on the CPU. B3: TensorRT FP16 on the GPU. B4: TensorRT INT8 on the GPU (implicit quantisation; a timing baseline, not an accuracy one). B3 and B4 are new; the earlier version had only CPU baselines because the installed onnxruntime has no CUDA provider.

Measurement. Every cell is 30 measured iterations after 5 warm-ups, wrapped in a tegrastats window stretched to at least 3 s so that power rests on dozens of samples; a cell that fails or times out is recorded as such. We report medians with p99, coefficient of variation, and maximum—never a median alone. Energy per inference is mean power times mean latency. Correctness is a bit-exact comparison of INT32 outputs against the lowering’s expected values, plus the normalised error of the dequantised output against FP32 onnxruntime.

Claims. Each number carries a provenance mark: ^M measured on hardware, ^E produced by the virtual FPGA with an emulated link, ^C from the cost model. The SplitInfer NMC and pool rows of Table II are measured on the board (bitstream from commit 3b789dc, 30 iterations, bit-exact on every one); the link-bandwidth sweep of §VI is emulated, and its UART point is anchored by those measurements.

VI. RESULTS

A. E2: every path on one workload

Table II and Fig. 4 put every execution path on the DLRM at batch 1. Three things are worth reading off. First, the model fits in host memory, so it *should not* be disaggregated, and the placer says so: with a normal host budget it places all 36 nodes on the host (^C). The SplitInfer rows exist to price the mechanism, not to recommend it here. Second, the INT8 host path is 1.64 ms—slower than the FP16 GPU’s 0.448 ms because it is a scalar CPU kernel; the interesting comparison is between the three SplitInfer modes, which share that kernel. Third, on the board NMC costs 546 ms per inference (30/30 bit-exact, p99 610 ms) and a cold pool with 64-page prefetch 309,573 ms (870 pages faulted): a 570× gap between the two ways of using the same memory node over the same link, with identical outputs. The virtual FPGA predicted 551 ms and 307,666 ms for the same two runs—within 1% of silicon—which is the evidence that the emulated sweep in §VI can be trusted away from the measured point.

Weight staging is a first-class cost. Before any inference the 3.6 MiB image must reach DDR2; at UART speed that takes 334 s on the board (334 s emulated). The previous version of this work excluded it from every reported latency. It is reported here as preparation time and, for the streaming case (§VI-C), inside the per-inference latency where it belongs.

TABLE II

E2: DLRM, BATCH 1, 30 ITERATIONS. LATENCY PER INFERENCE; LINK TRAFFIC PER INFERENCE; BIT-EXACT AGREEMENT WITH EXPECTED INT32 OUTPUT. PROVENANCE: ^M MEASURED ON THE BOARD, ^E VIRTUAL FPGA WITH THE EMULATED LINK AT UART PARAMETERS (11,520 B/s, 5 ms RTT; THE BOARD'S CALIBRATED ROUND TRIP IS 8.0 ms). POWER FOR THE $n=1$ COLD-POOL RUN WAS NOT SAMPLED.

Path	Prov.	Median (ms)	p99 (ms)	CV (%)	Power (mW)	Energy (mJ)	Link out / in (B)	Msgs	Bit-exact
B1 onnxruntime FP32, CPU	^M	0.547	0.620	—	6,510	3.63	—	—	—
B2 onnxruntime INT8, CPU	^M	0.270	0.692	—	6,142	1.83	—	—	—
B3 TensorRT FP16, GPU	^M	0.448	1.57	58	6,036	3.01	—	—	—
B4 TensorRT INT8, GPU	^M	0.403	4.65	147	6,359	4.72	—	—	—
SI host (INT8, host-resident)	^M	1.64	1.65	0	6,003	9.85	0 / 0	0	3/3
SI NMC (compute → memory)	^M	546	610	4	5,272	2902.77	1,907 / 476	59	30/30
SI pool, cold, 64-pg prefetch ($n=1$)	^M	309,573	—	—	—	—	0 / 3,563,520	14	1/1
SI pool, warm (host-cached)	^M	2.98	3,452	334	5,273	1330.81	0 / 2,867	0	30/30
SI NMC, virtual FPGA	^E	551	552	0	—	—	1,907 / 476	59	3/3
SI pool, cold, virtual FPGA	^E	307,666	309,060	1	—	—	0 / 3,534,848	14	3/3

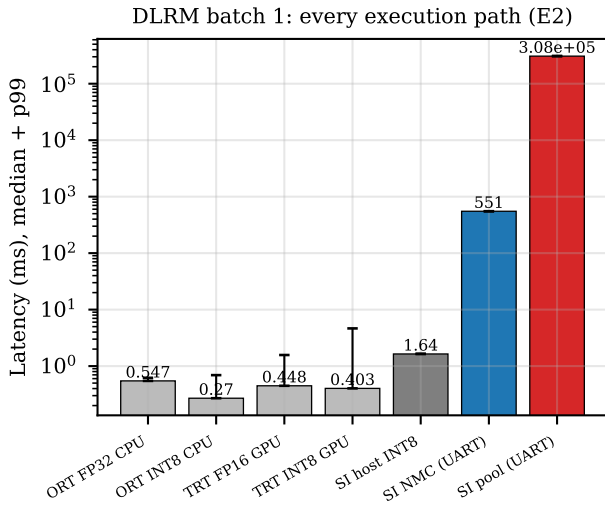


Fig. 4. E2 latencies (log scale; bar = median, whisker to p99).

B. E6: the crossover

Fig. 5 runs the same program in NMC and pool modes with the emulated link set to six classes from UART to PCIe 4×4 (Table III); the two starred points are the board measurements at the calibrated UART bandwidth, which sit on the emulated curves. NMC flattens at ~ 48.3 ms, the engines' compute time; the link only ever costs it 59 small messages. Pool falls with bandwidth until it meets the host-resident floor. The two cross between USB 2.0 (77.6 vs 135 ms, NMC wins) and USB 3 (54.1 vs 21.8 ms, pool wins): on this workload, moving compute to memory pays below roughly 10^8 bytes/s and stops paying above it. Equation (1) gives per-layer crossovers of 89 MB/s for the gathers and 196 MB/s for the 7 MB MatMul with the calibrated parameters—the right decade, bracketing the executed crossover.

Prefetch has a sign. At UART speed a 64-page prefetch is *slower* than demand paging (Table III, first row) even though it makes 14 round trips instead of 542. Each gather touches one 256-byte row of a 256 KB table; a 256 KB prefetch fetches the whole table. On a bandwidth-bound link the wasted bytes dominate; on a latency-bound link the saved round trips

DLRM: emulated-link sweep (E6) with board measurements

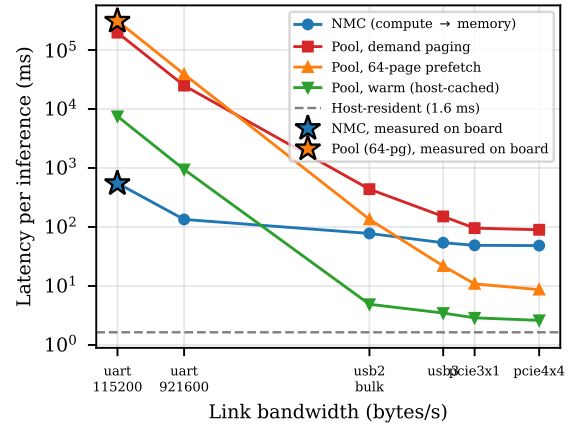


Fig. 5. E6: latency per inference versus link bandwidth for NMC and three pool configurations on the virtual FPGA (^E), with the board measurements (^M, stars) at the calibrated UART point. The crossover between NMC and prefetching pool lies between USB 2.0 and USB 3.

TABLE III

E6 (^E): MEDIAN LATENCY PER INFERENCE (MS) ACROSS LINK CLASSES.

Link	NMC	Pool $P=1$	Pool $P=64$	Pool $P=512$	Pool warm
UART 115.2k	551	196,769	310,512	326,422	7,500
UART 921.6k	134	24,935	38,829	40,810	943
USB 2.0 bulk	77.6	437	135	133	4.90
USB 3	54.1	151	21.8	17.5	3.48
PCIe 3 × 1	48.9	95.5	10.9	11.2	2.88
PCIe 4 × 4	48.3	89.7	8.68	6.26	2.61

dominate, and the order reverses from USB 2.0 upward. The right prefetch depth is a property of the layer's access pattern, which the placer knows and a page-fault handler does not.

Warm pool is not host-resident. With host copies retained across inferences, pool converges toward the host floor but never reaches it, because successive inputs select different embedding rows and each new row faults. For sparse layers the pool's steady-state cost is the input's locality, not the model's size.

TABLE IV
E7 (^E): 8-LAYER MLP, 167.6 MiB INT8 > 128 MB DDR2, STREAMED
THROUGH 2×23.9 MiB SLOTS.

Mode / link	Latency (ms)	of which streaming	Bit-exact
Host-resident	162	–	4/4
NMC, no link delay	124	58.4	4/4
NMC, USB 2.0 bulk	35,927	32,631	2/2
NMC, USB 3	13,188	9,905	2/2

C. E7: capacity, executed

An eight-layer 5000×5000 MLP has 167.6 MiB of INT8 weights—larger than the FPGA’s 128 MB. Lowered with streaming, its FC layers rotate through two 23.9 MiB slots and the runtime downloads each layer’s weights immediately before dispatching it; the peak DDR2 footprint is 48.0 MiB. The program runs end to end, bit-exact against the reference (4/4), with normalised error 0.082 against FP32. Its price is stated in Table IV: at USB 2.0 rates, 32,631 ms of every 35,927 ms inference is weight streaming; at USB 3, 9,905 ms of 13,188 ms. The capacity multiplier a streaming design earns is total weight divided by 2× the largest layer, and it is bought with W_{total}/B per inference. Over a UART link that is hours per inference; over PCIe it is tens of milliseconds. Whether it is worth buying is, again, a property of the link.

D. Cost model against execution

For the DLRM with host budget zero and resident weights, the placer chooses 33 NMC layers, 1 pool layer (a 4-byte bias, where one page fault beats one dispatch) and 2 host layers with no weights, and estimates 1,081 ms (^C); the executed NMC program runs in 551 ms (^E). The model’s dispatch and compute terms (358.1 and 37.8 ms) match execution to within 30%; its overestimate is the activation crossing it charges at the CONCAT (676.7 ms), which the lowering removes by pinning producers into the concatenation buffer. A placer on the lowered program rather than the ONNX graph would close that gap; we leave it as stated.

E. Accuracy

Per-tensor symmetric INT8 with the fused fixed-point epilogue reproduces FP32 outputs to a normalised error of 0.127 on the DLRM and 0.113 on the eight-layer MLP. The point of the scheme is not its accuracy but its determinism: host, pool and NMC agree to the bit.

VII. DISCUSSION

When does compute-to-memory win? Below the crossover—here 10^7 – 10^8 bytes/s—and only if the weights are already resident on the node; a design that must stream weights every inference pays W/B either way and NMC’s advantage collapses to the difference between engine and GPU compute time, which the GPU wins. Above the crossover, pool wins, and a node with no engines at all—a plain memory expander with a good link—is the right design. The value of NMC at the edge is therefore bounded by the slowness of edge links, which is a statement about today’s edge I/O, not about NMC.

What the UART substrate is for. It is the worst case, chosen so that every protocol cost is visible at millisecond scale. It is not proposed as a deployment. The sweep in Fig. 5 is the deployment guidance: it says which class of link makes which design correct.

Limitations. (1) Only the UART point is measured on silicon; the rest of the link sweep is emulated, with engine timing calibrated from simulation (the board’s FC layer ran at 0.56 MAC/cycle against 0.68 simulated). (2) The runtime executes one mode for a whole program; mixed per-layer placements are modelled, not executed. (3) Batch size is one; the MAC engine reuses activations across eight rows but not weights across samples. (4) INT8 per-tensor quantisation is the only precision. (5) Pool-mode compute runs on the CPU; a GPU pool path would add a host-to-device copy of the faulted pages, which the E2 GPU baselines show to be ~ 0.1 ms at this size.

VIII. RELATED WORK

Software DSM supplied the mechanism: Ivy [1] (sequentially consistent page-based DSM), Munin [2] (multiple consistency protocols), TreadMarks [3] (lazy release consistency). CXLWIN is a single-writer, release-consistent window with explicit ownership transfer—closest to Munin’s write-shared protocol with the release directed by a scheduler rather than by locks. **CXL** memory systems—Pond [5], TPP [6], DirectCXL [4]—assume a coherent link of $\sim 10^{10}$ bytes/s; this paper is about what happens three to six decades below that. **Weight streaming** for models larger than accelerator memory—FlexGen [7], ZeRO-Inference [8], PowerInfer [9]—moves weights to the GPU from host memory or SSD and overlaps it with compute; those systems are pool-mode designs at 10^9 – 10^{10} bytes/s and our results agree that pool is right there. **Near-memory and processing-in-memory**—HBM-PIM [10], UPMEM [11], and the survey of Ghose *et al.* [12]—place compute beside memory to avoid a bandwidth wall inside a package; we place it beside memory to avoid a bandwidth wall on a cable, and find the same rule with a different constant. **FPGA inference**—Vitis AI [13], FINN [14], hls4ml [15]—treats the FPGA as an accelerator; our engines are deliberately small, because the question is where compute should sit, not how fast an FPGA can be. **Edge partitioning**—Neurosurgeon [16], DADS [17], DeepThings [18]—partitions compute across devices by latency; our placer partitions *residency* under two memory constraints with three placements, and its distinguishing output is a crossover bandwidth per layer.

IX. CONCLUSION

For a model that does not fit an edge SoC, the choice between moving memory to compute and compute to memory is decided by the link. We built both over one commodity substrate—a page-granular load/store window with explicit ownership, and near-memory INT8 engines—made them execute the same program to the same integers, and measured the crossover by running it: below about 10^8 bytes/s the engines next to the memory win, above it the link is fast enough to

haul weights to the GPU, and a capacity-unlocking streaming design pays W/B per inference in either case. The artifact reproduces every number here from its data files, and the board measurements agree with the virtual FPGA to within 1%.

ACKNOWLEDGMENTS

The author used Claude Code (Anthropic) as a software-engineering assistant during the v2 rebuild. All design decisions, experiments and interpretations are the author's.

REFERENCES

- [1] K. Li and P. Hudak, "Memory coherence in shared virtual memory systems," *ACM Trans. Comput. Syst.*, vol. 7, no. 4, pp. 321–359, 1989.
- [2] J. B. Carter, J. K. Bennett, and W. Zwaenepoel, "Implementation and performance of Munin," in *Proc. SOSP*, 1991, pp. 152–164.
- [3] P. Keleher, A. L. Cox, S. Dwarkadas, and W. Zwaenepoel, "TreadMarks: Distributed shared memory on standard workstations and operating systems," in *Proc. USENIX Winter*, 1994, pp. 115–131.
- [4] D. Gouk *et al.*, "Direct access, high-performance memory disaggregation with DirectCXL," in *Proc. USENIX ATC*, 2022, pp. 287–304.
- [5] H. Li *et al.*, "Pond: CXL-based memory pooling systems for cloud platforms," in *Proc. ASPLOS*, 2023, pp. 574–587.
- [6] H. Al Maruf *et al.*, "TPP: Transparent page placement for CXL-enabled tiered memory," in *Proc. ASPLOS*, 2023, pp. 742–755.
- [7] Y. Sheng *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, 2023.
- [8] R. Y. Aminabadi *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.
- [9] Y. Song *et al.*, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.
- [10] S. Lee *et al.*, "Hardware architecture and software stack for PIM based on commercial DRAM technology," in *Proc. ISCA*, 2021, pp. 43–56.
- [11] J. Devaux, "The true processing in memory accelerator," in *Proc. Hot Chips*, 2019.
- [12] S. Ghose *et al.*, "Processing-in-memory: A workload-driven perspective," *IBM J. Res. Dev.*, vol. 63, no. 6, 2019.
- [13] Xilinx, "Vitis AI," 2023.
- [14] Y. Umuroglu *et al.*, "FINN: A framework for fast, scalable binarized neural network inference," in *Proc. FPGA*, 2017, pp. 65–74.
- [15] J. Duarte *et al.*, "Fast inference of deep neural networks in FPGAs for particle physics," *JINST*, vol. 13, no. 7, 2018.
- [16] Y. Kang *et al.*, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proc. ASPLOS*, 2017, pp. 615–629.
- [17] L. Zeng *et al.*, "DADS: Device-aware DNN splitting for edge inference," in *Proc. SEC*, 2020.
- [18] Z. Zhao *et al.*, "DeepThings: Distributed adaptive deep learning inference on resource-constrained IoT edge clusters," *IEEE TCAD*, vol. 37, no. 11, pp. 2348–2359, 2018.
- [19] M. Naumov *et al.*, "Deep learning recommendation model for personalization and recommendation systems," arXiv:1906.00091, 2019.

Hemraj Lamkuche is a researcher in heterogeneous computing and FPGA-based edge ML inference systems.